

编译器设计专题实验课2026

2026.05

实验五：语义分析框架

2018

- ▶ 实验五：SLR引导的语义分析框架实现
- ▶ 在SLR(1)语法分析的基础上，集成语义动作，构建抽象语法树（AST）并维护符号表，完成类型检查和初步语义验证。
- ▶ 文法扩展-在产生式中嵌入语义动作；属性计算-实现属性值的传递（如表达式类型、变量地址等）；符号表管理-支持变量声明、作用域嵌套和类型查询；错误检测-报告重复声明、类型不匹配等语义错误。

一、实验五：语义分析框架

实验五

- ▶ 实验五：SLR引导的语义分析框架实现
- ▶ 输入：-SLR(1)分析表（实验四输出）嵌入语义动作的文法规则（文件或硬编码）
- ▶ -Token流（实验二输出）
- ▶ 输出：-抽象语法树（AST）的可视化表示，符号表（变量名、类型、作用域），语义错误报告（如类型错误）
- ▶ 在SLR(1)语法分析的基础上，集成语义动作，构建抽象语法树（AST）并维护符号表，完成类型检查和初步语义验证。



实验五：语义分析框架



产生式	语义规则
$S \rightarrow id = E$	$S.code = E.code$ $\parallel gen(top.get(id.lexeme) '=' E.addr)$
$E \rightarrow E_1 + E_2$	$E.addr = new Temp();$ $E.code = E_1.code \parallel E_2.code$ $\parallel gen(E.addr := 'E_1.addr '+'E_2.addr')$
$E \rightarrow E_1 * E_2$	$E.addr = new Temp();$ $E.code = E_1.code \parallel E_2.code$ $\parallel gen(E.addr := 'E_1.addr '*'E_2.addr')$
$E \rightarrow (E_1)$	$E.addr = E_1.addr;$ $E.code = E_1.code$
$E \rightarrow id$	$E.addr = top.get(id.lexeme);$ $E.code = ''$

一、实验五：语义分析框架

思源堂

▶ 实验六、七八的安排和输入输出

实验阶段	输入	输出	核心任务
实验六	AST、符号表	中间代码（三地址码、四元式或LLVM IR）	语法树翻译为中间表示
实验七	中间代码、符号表	内存地址映射文件	内存地址分配与代码序列化
实验八	中间代码、内存映像、符号表	汇编/可执行代码	目标代码生成与优化

第七次课程开始验收，同学们可以参考思源学堂的示例代码开始准备。验收阶段从源程序阶段到中间代码或者目标代码阶段。



二、实验步骤和结果

- ▶ 实验步骤：
- ▶ 文法扩展：为每个产生式添加语义动作（例如生成AST节点或更新符号表）；
- ▶ 属性栈设计：在语法分析栈旁维护属性栈，保存每个符号的语义值（如AST节点指针、类型）
- ▶ 语义动作执行：规约时调用对应的语义动作函数，操作属性栈和符号表。
- ▶ 符号表实现：支持作用域栈结构，处理变量声明和引用。
- ▶ 错误处理：在语义动作中检测错误（如未声明变量）并记录。



二、实验步骤和结果

```
语法错误: 状态 0 无法处理符号 $B$WORK
root@dsw-544537-86b6854df5-k9849:/mnt/workspace# g++ -std=c++17 -I /usr/include/nlohmann/ semantic.cpp -o semantic
root@dsw-544537-86b6854df5-k9849:/mnt/workspace# ./semantic
请输入SLR分析表文件: SLR1.txt
请输入Token流文件: token.txt
terminate called after throwing an instance of 'nlohmann::detail::parse_error'
  what():  [json.exception.parse_error.101] parse error at line 2, column 37: syntax error while parsing object
literal; last read: '"s2"❖; expected '}'
已中止
```

三、实验六：中间代码生成

赵永亮

- ▶ 实验六：中间代码生成
- ▶ 输入抽象语法树信息或其他，token，源程序等等。
- ▶ 输出中间代码表示，三元式、四元式、三地址方式均可，以下输出均可

```
input x;  
if 0 < x then  
    fact = 1  
else fact=2;  
repeat  
    fact = fact * x;  
    x = x - 1  
until x == 0;  
print fact
```

```
INPUT x  
t1 = 0  
IF t1 < x THEN I1 ELSE I2  
LABEL I1  
fact = 1  
GOTO I3  
LABEL I2  
fact = 2  
LABEL I3  
fact = fact * x  
x = x - 1  
IF x == 0 THEN I4 ELSE I3  
LABEL I4  
PRINT fact
```

例如 $X = a*b+c/d$ 的四元式序列：

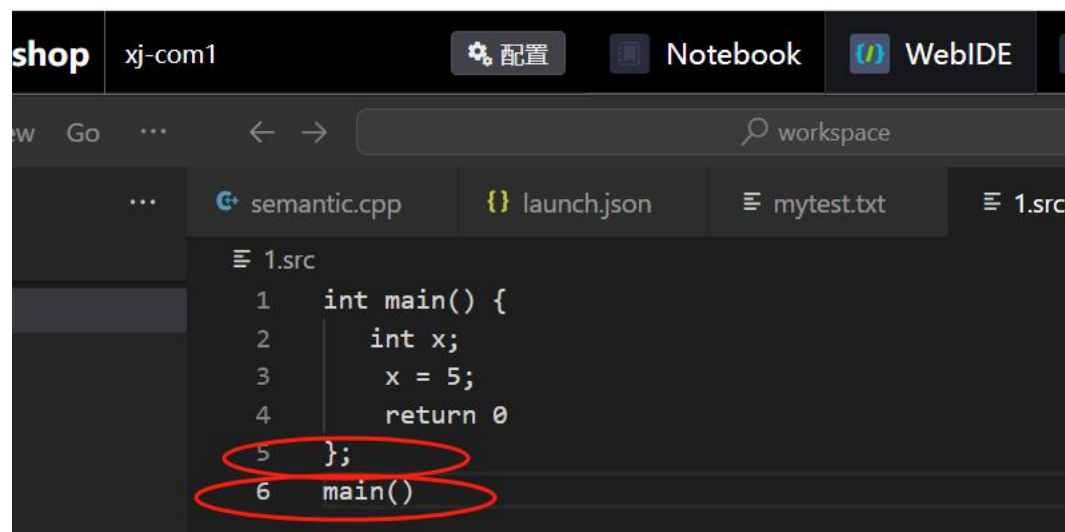
1. $(*, a, b, T1)$
2. $(/, c, d, T2)$
3. $(+, T1, T2, T3)$
4. $(=, T3, -, X)$

赵永亮

三、实验六：中间代码生成

李永亮

- ▶ 实验六：中间代码生成
- ▶ 示例代码1



```
shop xj-com1 配置 Notebook WebIDE  
workspace  
semantic.cpp launch.json mytest.txt 1.src  
1 src  
1 int main() {  
2     int x;  
3     x = 5;  
4     return 0  
5 };  
6 main()
```

- ▶ 示例代码库已经上传，根据同学的修正，目前正确代码大多数，错误代码非常少，集中在词法分析阶段，能够减轻同学在出错处理阶段精力，主要用于分析文法。

李永亮

四、示例代码库和文法的解释

李永亮

词法规则文件

d ~ (ID, name)↵

ID = [a-z][a-z0-9]*↵

i ~ (INT, value)↵

NUM = [+]?[0-9]+↵

f ~ (FLO, value)↵

FLO = [+]?[0-9]*.[0-9]+ | [+]?[0-9]+.[0-9]*↵

+ ~ (ADD, -)↵

ADD = {+}↵

* ~ (MUL, -)↵

MUL = {*} ↵

r ~ (ROP, name)↵

ROP = {<, <=, ==}↵

\= ~ (ASG, -)↵

ASG = {\=}↵

\(~ (LPA, -)↵

LPA = {\(} ↵

\) ~ (RPA, -)↵

RPA = {\)}↵

[~ (LBK, -)↵

LBK = {[}↵

] ~ (RBK, -)↵

RBK = {]}↵

\{ ~ (LBR, -)↵

LBR = {\{ }↵

\} ~ (RBR, -)↵

RBR = {\} }↵

\. ~ (CMA, -)↵

CMA = {\. }↵

\: ~ (SCO, -)↵

SCO = {\: }↵

int ~ (INT, -)↵

float ~ (FLOAT, -)↵

if ~ (IF, -)↵

else ~ (ELSE, -)↵

while ~ (WHILE, -)↵

return ~ (RETURN, -) ↵

input ~ (INPUT, -)↵

print ~ (PRINT, -)↵

void ~ (VOID, -)↵

李永亮

四、示例代码库和文法的解释

2018

词法类别和正则（附录A）

ID = L([a-z][a-z0-9]*)\KEY

KEY = {int, float, void, if, else, while, return, input, print}

INT = {int}; FLOAT = {float}; VOID = {void}; IF = {if}; ELSE = {else}

WHILE = {while}; RETURN = {return}; INPUT = {input}; PRINT = {print}

NUM = L([+ -]?[0-9]*)

FLO = L([+ -]?[0-9]*.[0-9]+|[+ -]?[0-9]+.[0-9]*)

OP = AOP $\cup$ ROP $\cup$ BOP $\cup$ {=, +=, ++}

AOP = {+, -, *, /}

ADD = {+}; SUB = {-}; MUL = {*}; DIV = {/}

ROP = {<, <=, ==, >, >=, !=}

BOP = {^, v, |} 或 BOP = {&&, ||, !}

ASG = {=}; AAS = {+=}; AAA = {++}

DLI = {(, [, \{, \}, \, _}

LPA = {(}; RPA = {)}; LBK = {[}; RBK = {]}; LBR = {\{ }; RBR = {\} }; CMA
= {\, }, SCO = {;}

d:ID; int:KEY; int:INT; i:NUM; f:FLO; o:AOP; +:ADD; r:ROP; <:L(<);

^:BOP; ^:L(^); &&:BOP; (:DLL;):LPA;

2018

四、示例代码库和文法的解释

王守志

▶ 二者区别

两份规则的关系：附录A中的词法是功能更完整、定义更严谨的超集或升级版。

主要区别：

标识符定义：附录A中的词法通过 \KEY 操作排除了关键字，避免了词法分析冲突，而第一个词法规则没有，

运算符完整性：第一个词法规则严重缺失关系运算符（如 >、!=）、赋值运算符和逻辑运算符，无法描述完整程序。附录A中的词法定义了完整的运算符集。

请注意数值定义：规则B的整数定义（ $[0-9]^*$ ）可能匹配空字符串。

四、示例代码库和文法的解释

王元超

主文法

$$P \rightarrow \check{D} \check{S} \leftarrow$$
$$\check{D} \rightarrow \varepsilon \mid \check{D} \underline{D}_i \leftarrow$$
$$D \rightarrow T d \mid T d[i] \mid T d(\check{A}) \{\check{D} \check{S}\} \leftarrow$$
$$T \rightarrow \text{int} \mid \text{float} \mid \text{void} \leftarrow$$
$$\check{A} \rightarrow \varepsilon \mid \check{A} \underline{A}_i \leftarrow$$
$$A \rightarrow T d \mid T \underline{d}[] \mid T d(T) \leftarrow$$
$$\check{S} \rightarrow S \mid \check{S}_i S \leftarrow$$
$$S \rightarrow d = E \mid d[E] = E \mid \text{if} (B) S \mid \text{if} (B) S \text{ else } S \mid \text{while} (B) S \mid \text{return } E \mid \{\check{S}\} \mid d(\check{R}) \mid \cdot$$
$$E \rightarrow \underline{i} \mid f \mid d \mid d[E] \mid E + E \mid E * E \mid (E) \mid d(\check{R}) \leftarrow$$
$$B \rightarrow E r E \mid E \leftarrow$$
$$\check{R} \rightarrow \varepsilon \mid \check{R} \underline{R}_i \leftarrow$$
$$R \rightarrow E \mid \underline{d}[] \mid d() \leftarrow$$

- ▶ 每个变元都指代一种语法单元: 程序 P , 声明表 $\check{D}$, 声明 D , 类型 T , 整数表 $\check{I}c$, 形参表 $\check{A}c$, 形参 A , $\check{R}$ 实参表, 实参 R , 语句表 $\check{S}c$, 语句 S , 表达式表 $\check{E}c$, 表达式 E , 布尔表达式 B 。



- ▶ 写成可配置的CFG，程序的输入文件

```
Prog -> DeclList
DeclList -> DeclList Decl | Decl
Decl -> VarDecl | FunDecl
VarDecl -> Type ID SEMI | Type ID LBRACK INT_NUM RBRACK SEMI | Type ID ASG Expr SEMI
Type -> INT | FLOAT | VOID
FunDecl -> Type ID LPAR ParamList RPAR CompStmt
ParamList -> ParamList COMMA Param | Param | ε
Param -> Type ID | Type ID LBRACK RBRACK
CompStmt -> LBR StmtList RBR
StmtList -> StmtList Stmt | ε
Stmt -> VarDecl | OtherStmt
OtherStmt -> ExprStmt | CompStmt | IfStmt | LoopStmt | RetStmt | PrintStmt
PrintStmt -> PRINT LPAR Expr RPAR SEMI
ExprStmt -> Expr SEMI | SEMI
IfStmt -> IF LPAR Expr RPAR CompStmt | IF LPAR Expr RPAR CompStmt ELSE CompStmt
LoopStmt -> WHILE LPAR Expr RPAR Stmt
RetStmt -> RETURN Expr SEMI | RETURN SEMI
Expr -> ID ASG Expr | ID LBRACK Expr RBRACK ASG Expr | ID LPAR ArgList RPAR | SimpExpr
SimpExpr -> AddExpr REL_OP AddExpr | AddExpr
AddExpr -> AddExpr ADD Term | Term
Term -> Term MUL Fact | Fact
Fact -> ID | ID LBRACK Expr RBRACK | INT_NUM | FLOAT_NUM | LPAR Expr RPAR
ArgList -> ArgList COMMA Expr | Expr | ε
```

四、示例代码库和文法的解释

▶ 示例代码库提供若干初级程序

▶ 程序段

$$1. P \rightarrow \check{D}\check{S}$$

▶ $\check{D}$ 是变量的声明语句集. $\check{S}$ 是非变量声明语句集, 包括赋值语句

单字母终结符指代一种词法单元:

标识符d, 整数i, 实数f, 算术运算符o, 关系运算符r, 单目算术运算符u。

此外的终结符都是与自身对应的一个词法单元, 这些词法单元包含了关键字、分隔符、括号、逻辑运算符。每个词法单元都是某个词法类别(种属)的一个实例, 采用二元组(<种属>, <个体>)表示。

1.src - 记事本

文件(F) 编辑(E) 格式(O) 查看

```
int main() {
    int x;
    x = 5;
    return 0
};
main()
```

函数声明
变量声明
赋值语句
return语句

关于分号的用法
S->语句列表之间用分号, 而语句结束无需分号
因此return 0 没有分号, 函数声明结束部分有分号

2.src - 记事本

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

```
void func() {
    int a = 3;
    if (a && 5) {
        a = 10;
    }
};
func()
// 错误: 文法不支持布尔操作符"&&"
```

5.src - 记事本

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

```
void test() {
    print a
};
test()
```

四、示例代码库和文法的解释

▶ 示例代码库和文法的解释

▶ 此外值得注意的本文法的实参和形参部分

▶ $A \rightarrow Td()$

▶ A 代表函数参数列表的声明. 可以是普通变量参数 $A \rightarrow Td$, 可以是数组参数 $A \rightarrow Td[]$, 也可以是函数参数 $A \rightarrow Td()$

$\tilde{A} \rightarrow \varepsilon \mid \tilde{A} A; \quad \tilde{R} \rightarrow \varepsilon \mid \tilde{R} R;$

▶ $\tilde{A}$ 是函数参数列表的声明集. A 代表函数参数列表中某个参数的声明.

▶ 参数之间用分号隔开, 而不是逗号, 即使是最后一个参数也需要使用

▶ 分号结尾.

```
void func1(int abc; int xyz; int arr[]; int func2();)
```

▶ R 是函数调用参数集. R 是函数参数列表中某个参数

▶ 的调用. 参数之间用逗号隔开, 最后一个参数需要逗号

▶ 例如

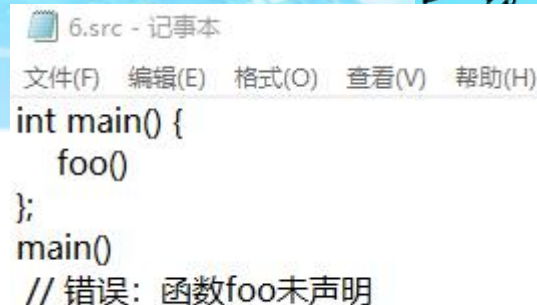
```
26.src - 记事本
文件(F) 编辑(E) 格式(O) 查看(V)
int add(int a; int b;) {
    return a + b
};

int calc() {
    int sum;
    sum = add(3, 4);
    return sum
};
calc()
```

四、示例代码库和文法的解释

▶ 示例代码库和词法的解释

- 程序的main被识别为ID，不是关键字
- 关键字只有int, float, void, if, else, while, return, input 和print,
- 没有 for,除了分号;和逗号，不支持双引号、单引号、\\等其他界符。
- 没有注释
- 升级之后的程序支持与或非等操作
- 支持，{[(,)]}; ,
- 验收程序可以支持部分示例文法和示例词法，不需要完全一致。



6.src - 记事本

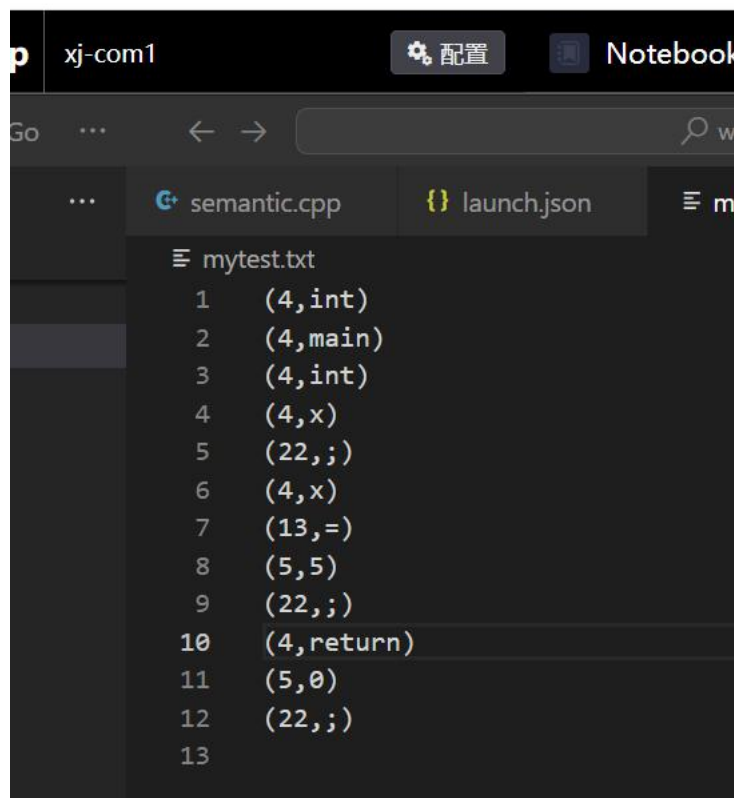
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

```
int main() {  
    foo()  
};  
main()  
// 错误: 函数foo未声明
```

四、示例代码库和文法的解释

李永亮

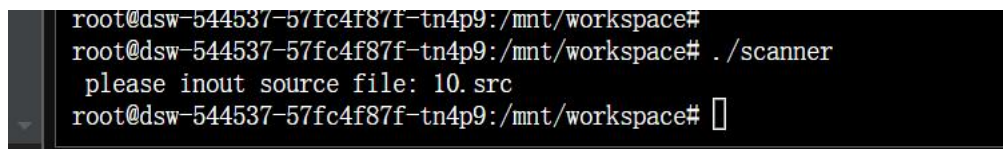
- ▶ 词法部分
- ▶ 示例程序1.src经过词法以后得到token序列



A screenshot of a code editor window titled 'xj-com1'. It shows a file named 'mytest.txt' with 13 lines of token sequences. The tokens are pairs of a number and a string, representing tokens from a scanner. The editor also shows tabs for 'semantic.cpp' and 'launch.json'.

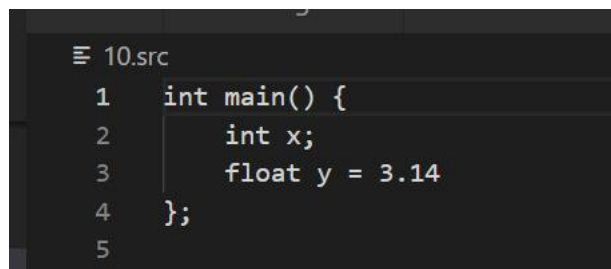
```
1 (4,int)
2 (4,main)
3 (4,int)
4 (4,x)
5 (22,;)
6 (4,x)
7 (13,=)
8 (5,5)
9 (22,;)
10 (4,return)
11 (5,0)
12 (22,;)
13
```

10.src



A terminal window showing the execution of a scanner program. The user runs './scanner' and the program prompts for a source file. The user enters '10.src'.

```
root@dsw-544537-57fc4f87f-tn4p9:/mnt/workspace#
root@dsw-544537-57fc4f87f-tn4p9:/mnt/workspace# ./scanner
please inout source file: 10.src
root@dsw-544537-57fc4f87f-tn4p9:/mnt/workspace#
```



A screenshot of a code editor showing the source code for '10.src'. It is a C program with a main function that declares an integer 'x' and a float 'y' with the value 3.14.

```
1 int main() {
2     int x;
3     float y = 3.14
4 };
5
```

李永亮

四、示例代码库和文法的解释

李兆亮

- ▶ 词法部分
- ▶ 示例程序10.src经过词法以后得到token序列

```
≡ mytest.txt
1  (4,int)
2  (4,main)
3  (4,int)
4  (4,x)
5  (22,;)
6  (4,float)
7  (4,y)
8  (13,=)
9  (5,3)
10 (16,.)
11 (5,14)
12 (22,;)
13
```

$E \rightarrow i | f | d | d[E] | E + E | E * E | (E) | d(\check{R})$

- 表达式可以是i（整数），f（浮点数），d（某标识符）
- E+E，E*E，或者（），或者d（R）-函数调用表达式，或者d（E）（这个文法，大作业不支持，也可以不写）

四、示例代码库和文法的解释

四、示例代码库和文法的解释

实验六输入

```
int main ()
{
    a=1;
    b=2;
    c=1+2*3+(4+5)*6;

    if a+b<c
    {
        a=a+b;
        b=b+a;
    }
    else
        c=1000;
}
```

实验六输出

```
workspace
nner-back.cpp SLR1.cpp mytestResult.txt X
mytestResult.txt
1 1. (=, 1, , a)
2 2. (=, 2, , b)
3 3. (*, 2, 3, T1)
4 4. (+, 1, T1, T2)
5 5. (+, 4, 5, T3)
6 6. (*, T3, 6, T4)
7 7. (+, T2, T4, T5)
8 8. (=, T5, , c)
9 9. (+, a, b, T6)
10 10. (j<, T6, c, 12)
11 11. (j, , , 17)
12 12. (+, a, b, T7)
13 13. (=, T7, , a)
14 14. (+, b, a, T8)
15 15. (=, T8, , b)
16 16. (j, , , 18)
17 17. (=, 1000, , c)
```

- 输入是一段符合文法的代码
- 输出是四元式方式
- 本次实验结果可作为验收结果